📚 جزوه سیستم عامل | فصل ۹: حافظه اصلی

📝 بخش ۴: ساختار جدول صفحه (Structure of the Page Table)

۱. مقدمه: چرا به ساختارهای عجیب نیاز داریم؟ 🤯

در سیستم‌های مدرن، فضای آدرس منطقی خیلی بزرگ است (مثلاً 2^{32} یا 2^{64} بایت).

- فرض کنید یک سیستم ۳۲ بیتی با صفحات ۴ کیلوبایتی (2^{12}) داریم.
- تعداد صفحات برابر است با: $2^{32} / 2^{12} = 2^{20}$ (یعنی ۱ میلیون صفحه!).
- اگر هر ردیف جدول ۴ بایت باشد، کل جدول صفحه **۴ مگابایت** می‌شود.
- **مشکل:** ما نمی‌توانیم برای هر پردازه، ۴ مگابایت فضای پیوسته در RAM فقط برای جدول صفحه اشغال کنیم! (اگر ۱۰۰ پردازه داشته باشیم، ۴۰۰ مگابایت رم فقط برای جدول‌ها هدر می‌رود).

راه‌حل: باید ساختار جدول را تغییر دهیم تا فضای کمتری بگیرد یا مدیریت آن آسان‌تر شود.

۲. تکنیک اول: صفحه‌بندی سلسله‌مراتبی (Hierarchical Paging) 🌳

ایده: "بیایید خودِ جدول صفحه را هم صفحه‌بندی کنیم!" یعنی به جای یک لیست دراز و پیوسته، جدول را تکه‌تکه کنیم و آدرس این تکه‌ها را در یک "جدولِ جدول‌ها" نگه داریم.

صفحه‌بندی دوسطحی (Two-Level Paging)

در سیستم ۳۲ بیتی، آدرس منطقی به ۳ بخش تقسیم می‌شود:

1. p_1 **(۱۰ بیت):** ایندکس جدول بیرونی (Outer Page Table).
2. p_2 **(۱۰ بیت):** ایندکس جدول داخلی (Inner Page Table).
3. d **(۱۲ بیت):** افست صفحه (Offset).

نحوه کار:

- p_1 به ما می‌گوید کدام "تکه" از جدول اصلی را نیاز داریم.

- 2_p مکان دقیق صفحه را در آن تکه پیدا می‌کند.
- **مزیت:** لازم نیست کل جدول اصلی در حافظه باشد؛ فقط تکه‌هایی که نیاز داریم بارگذاری می‌شوند.
- **نکته:** برای سیستم‌های ۶۴ بیتی، ۲ سطح کافی نیست (جدول بیرونی باز هم خیلی بزرگ می‌شود). باید از ۳ یا ۴ سطح استفاده کرد که دسترسی به حافظه را کند می‌کند.

۳. تکنیک دوم: جدول صفحات درهم (Hashed Page Tables) #🔲

ایده: به جای استفاده از آرایه (ایندکس مستقیم)، از **جدول درهم‌سازی (Hash Table)** استفاده کنیم. این روش برای فضاهای آدرس بزرگ (بیشتر از ۳۲ بیت) عالی است.

نحوه کار:

1. شماره صفحه مجازی (p) را به یک تابع Hash می‌دهیم.
2. تابع Hash یک آدرس تولید می‌کند که به یک خانه از جدول اشاره دارد.
3. هر خانه جدول شامل یک **لیست پیوندی** از عناصر است (برای مدیریت برخوردها).
4. در این لیست جستجو می‌کنیم تا شماره صفحه (p) را پیدا کنیم و قاب متناظر (f) را برداریم.

مزیت: اندازه جدول به تعداد صفحات *استفاده شده* بستگی دارد، نه کل فضای آدرس.

۴. تکنیک سوم: جدول صفحات وارونه (Inverted Page Tables) 🙃

ایده: "چرا برای هر پردازه یک جدول جدا داشته باشیم؟ بیایید **یک جدول کل برای کل سیستم** داشته باشیم."

در روش‌های قبلی، جدول بر اساس "آدرس مجازی" مرتب شده بود (صفحه ۱، صفحه ۲، ...). در اینجا، جدول بر اساس **"آدرس فیزیکی" (قاب‌ها)** مرتب می‌شود.

- به ازای هر **قاب فیزیکی** در RAM، دقیقاً **یک ردیف** در جدول داریم.
- هر ردیف می‌گوید: "این قاب فیزیکی الان دستِ کدام پردازه (PID) و کدام صفحه (Page Number) است؟"

نحوه ترجمه:

- باید در کل جدول جستجو کنیم تا ببینیم (PID, p) کجاست. ایندکس آن ردیف (i) می‌شود همان آدرس فیزیکی!
- **مزیت:** صرفه‌جویی عظیم در حافظه (چون فقط یک جدول برای کل سیستم داریم، نه برای هر پردازه).
- **عیب بزرگ:** جستجو در جدول کُند است (چون مرتب نیست).
  - *راه‌حل:* استفاده از یک جدول Hash برای سرعت بخشیدن به جستجو.
- **مشکل اشتراک:** پیاده‌سازی "حافظه مشترک" (Shared Memory) در این روش سخت است (چون هر قاب فیزیکی فقط می‌تواند به یک آدرس مجازی اشاره کند).

۵. مبادله (Swapping) 🔄

- اگر RAM پر شد چه کنیم؟
- سیستم‌عامل پردازه‌هایی که فعلاً کار نمی‌کنند (Idle) را موقتاً به دیسک سخت (که به آن **Backing Store** می‌گویند) می‌فرستد (**Swap Out**).
- وقتی دوباره نوبتشان شد، به حافظه برگردانده می‌شوند (**Swap In**).
- این کار اجازه می‌دهد تعداد برنامه‌های باز، بیشتر از گنجایش واقعی RAM باشد.

📊 جدول جمع‌بندی (مخصوص شب امتحان)

تکنیک	مشکل اصلی که حل می‌کند	روش کار خلاصه	مزیت/عیب
سلسله‌مراتبی	جدول‌های بزرگ و پیوسته.	جدولِ جدول‌ها (درختی).	✅ انعطاف‌پذیر ❌ دسترسی حافظه زیاد (کند).
درهم‌سازی (Hashed)	فضای آدرس‌های خیلی بزرگ (64 بیتی).	استفاده از Hash Function.	✅ مناسب سیستم‌های ۶۴ بیتی.
وارونه (Inverted)	مصرف زیاد حافظه توسط جدول‌ها.	یک جدول برای کل سیستم (بر اساس فیزیک).	✅ کم‌مصرف‌ترین حافظه ❌ جستجوی کند / حافظه مشترک سخت.